

SECURITY ANALYSIS REPORT

Project Title: Staking DApp Security Analysis Report

Technology Stack: Solidity, React.js, Ethers.js, MetaMask, Vercel

1. Introduction

This report presents a security analysis of the Staking DApp developed using Solidity smart contracts and a React-based frontend. The application allows users to connect their MetaMask wallet, stake ETH, view their staked balance, and withdraw funds securely.

The objective of this security review is to identify potential vulnerabilities, evaluate existing security controls, and recommend improvements to enhance the overall security of the application.

2. Project Overview

The Staking DApp consists of two major components:

Smart Contract Layer

- Handles staking and withdrawal operations.
- Maintains user balances on the blockchain.
- Stores staking information securely.

Frontend Layer

- Built using React.js.
- Uses Ethers.js to communicate with the smart contract.
- Connects to MetaMask for wallet authentication.

Core Functionalities

1. Connect Wallet
2. Stake ETH
3. Check Staked Balance
4. Withdraw ETH
5. View Transaction Status

3. Smart Contract Review

The smart contract contains the following functions:

Function	Purpose
stake()	Allows users to stake ETH
withdraw(uint256 amount)	Allows users to withdraw staked ETH
balances(address)	Returns balance from mapping
getBalance(address)	Returns user staking balance

SECURITY ANALYSIS REPORT

Balance Storage

```
mapping(address => uint256) public balances;
```

This mapping ensures that each user has an independent staking balance.

4. Security Analysis

4.1 Access Control

Observation

Users can only access and withdraw their own funds.

Security Check

```
balances[msg.sender]
```

Result

PASS

Risk Level

Low

Impact

Prevents unauthorized users from withdrawing funds belonging to other users.

4.2 Input Validation

Observation

The contract verifies that users stake a non-zero amount.

Security Check

```
require(msg.value > 0, "Send ETH");
```

Result

PASS

Risk Level

Low

Impact

Prevents invalid or empty staking transactions.

4.3 Balance Verification

Observation

Before processing withdrawals, the contract validates the user's available balance.

SECURITY ANALYSIS REPORT

Security Check

```
require(  
    balances[msg.sender] >= amount,  
    "Insufficient balance"  
);
```

Result

PASS

Risk Level

Low

Impact

Prevents users from withdrawing more funds than they have staked.

4.4 Reentrancy Analysis

Observation

The contract updates internal balances before transferring ETH.

Security Check

```
balances[msg.sender] -= amount;  
payable(msg.sender).transfer(amount);
```

Result

PASS

Risk Level

Low

Impact

Reduces the possibility of reentrancy attacks.

Recommendation

Use OpenZeppelin ReentrancyGuard for additional protection.

4.5 Arithmetic Safety

Observation

The project uses Solidity version 0.8.x.

Security Benefit

Solidity 0.8.x automatically checks for:

- Integer Overflow
- Integer Underflow

SECURITY ANALYSIS REPORT

Result

PASS

Risk Level

Low

4.6 Frontend Security

Observation

Wallet authentication is handled by MetaMask.

Security Benefits

- No passwords stored
- No private keys stored
- Secure transaction approval process
- User-controlled signing

Result

PASS

Risk Level

Low

5. Vulnerability Assessment

Vulnerability	Severity	Status
Unauthorized Withdrawal	Low	Mitigated
Invalid Staking Amount	Low	Protected
Balance Manipulation	Low	Protected
Reentrancy Attack	Low	Partially Mitigated
Integer Overflow	Low	Protected
Private Key Exposure	Low	Protected

6. Security Recommendations

To further improve the security of the project, the following enhancements are recommended:

Recommendation 1

Implement OpenZeppelin ReentrancyGuard.

Recommendation 2

Add event logging for:

- Stake Transactions
- Withdraw Transactions

SECURITY ANALYSIS REPORT

Example:

```
event Staked(address indexed user, uint256 amount);  
event Withdrawn(address indexed user, uint256 amount);
```

Recommendation 3

Implement staking reward calculations.

Recommendation 4

Add transaction history tracking.

Recommendation 5

Perform third-party smart contract audits before production deployment.

7. Testing Summary

The following tests were successfully performed:

Test Case	Status
Wallet Connection	Passed
ETH Staking	Passed
Balance Retrieval	Passed
ETH Withdrawal	Passed
Contract Deployment	Passed
Frontend Integration	Passed

Test Environment

- Remix IDE
- MetaMask Wallet
- Ethereum-Compatible Network
- React + Vite Frontend
- Ethers.js Integration

8. Conclusion

The security analysis of the Staking DApp indicates that the application implements fundamental security measures required for a basic decentralized staking platform. The contract validates user inputs, prevents unauthorized withdrawals, and safely manages user balances.

No critical vulnerabilities were identified during testing. The project demonstrates secure interaction between the frontend and blockchain while following essential smart contract development practices.

Future enhancements such as event logging, reward mechanisms, and OpenZeppelin security modules can further strengthen the application for production-level deployment.